The University of Texas at Austin
Aerospace Engineering
and Engineering Mechanics
Cockrell School of Engineering

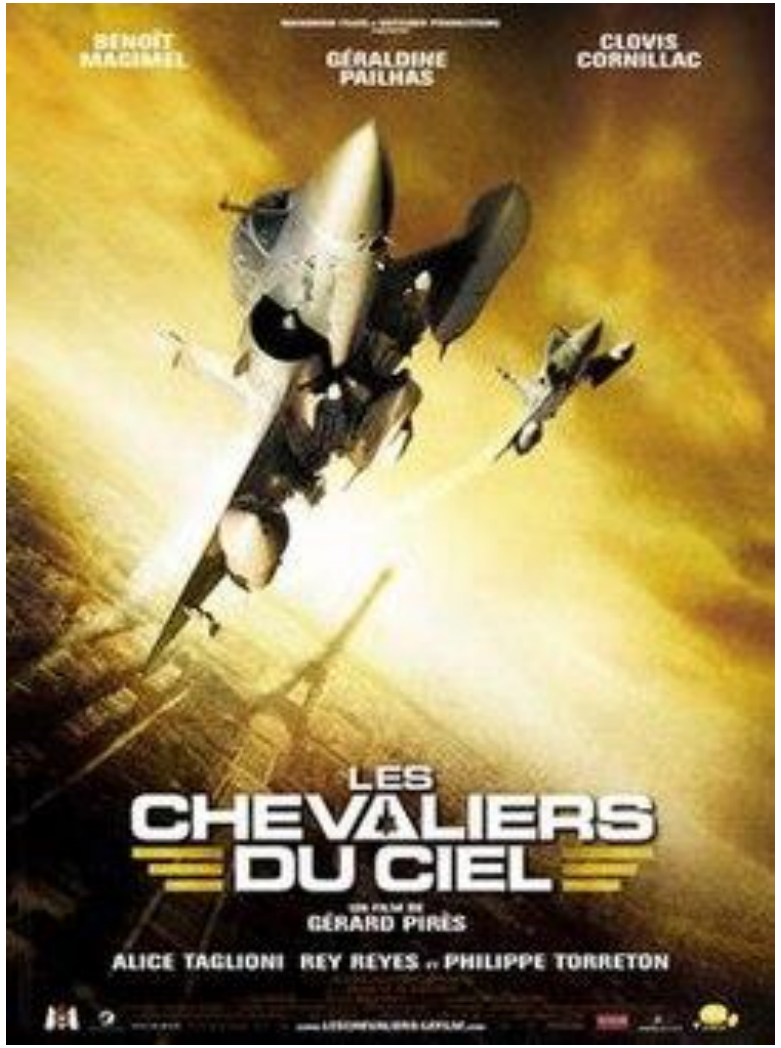
Apr.27. 2026

OBSERVING SATELLITE PROXIMITY OPERATION

Designing Observer Satellite's Optimal Trajectory by Second-Order MDP with Dynamic Programming

Hongseok Kim
Graduate Research Assistant, The University of Texas at Austin

Les Chevaliers du ciel



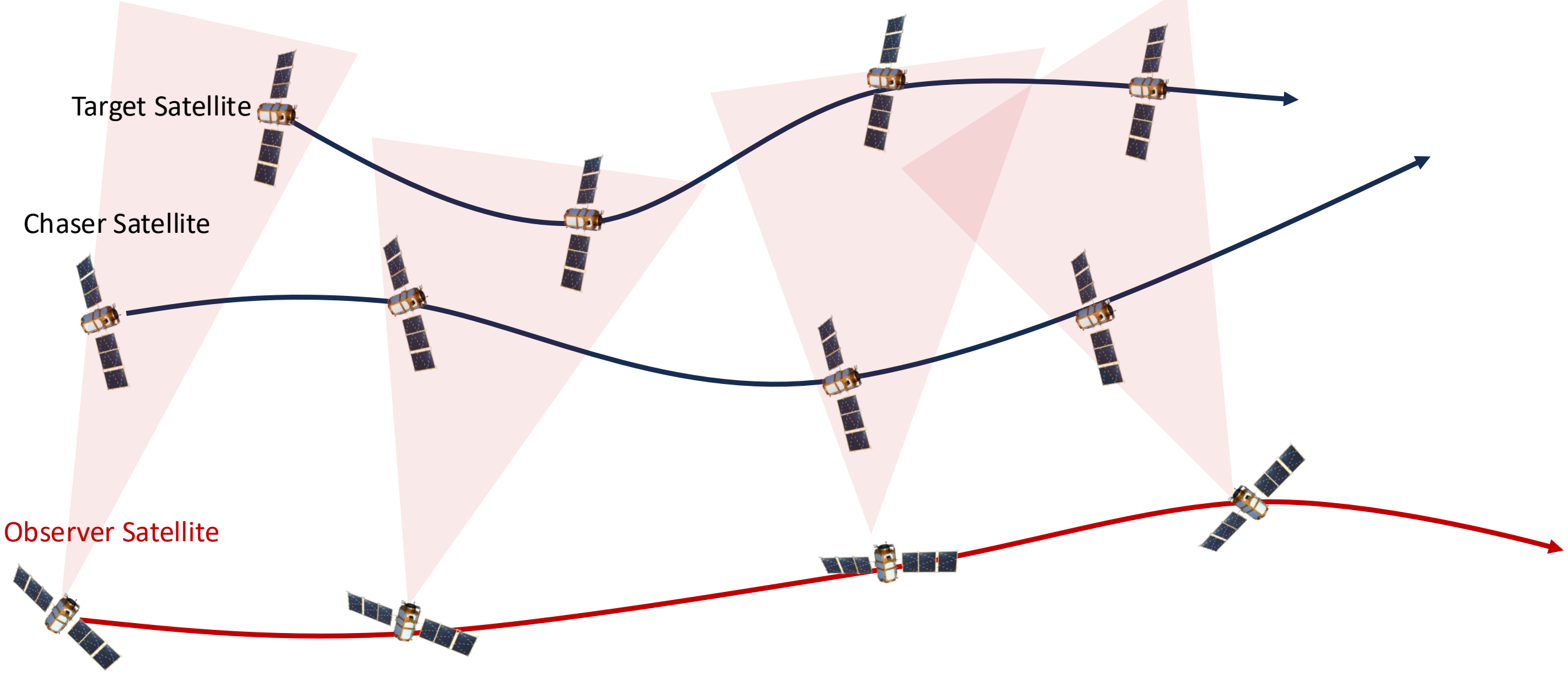
Les Chevaliers du Ciel (Sky fighters) was filmed in co-operation with the French Air Force. Initially the standard safety rules applied, but eventually the minimum allowed altitude was reduced to 3 m (10 ft) and the minimum distance between aircraft was reduced to 1 m (3 ft).

The filming of these flight sequences were mainly done from the air by Eric Magnan. To achieve this, one of the Mirage's external fuel tanks was modified to fit multiple cameras.

A HD camera was considered for this purpose, but it did not fit in the fuel pod. Tracking shots were done from a hired Lear Jet.

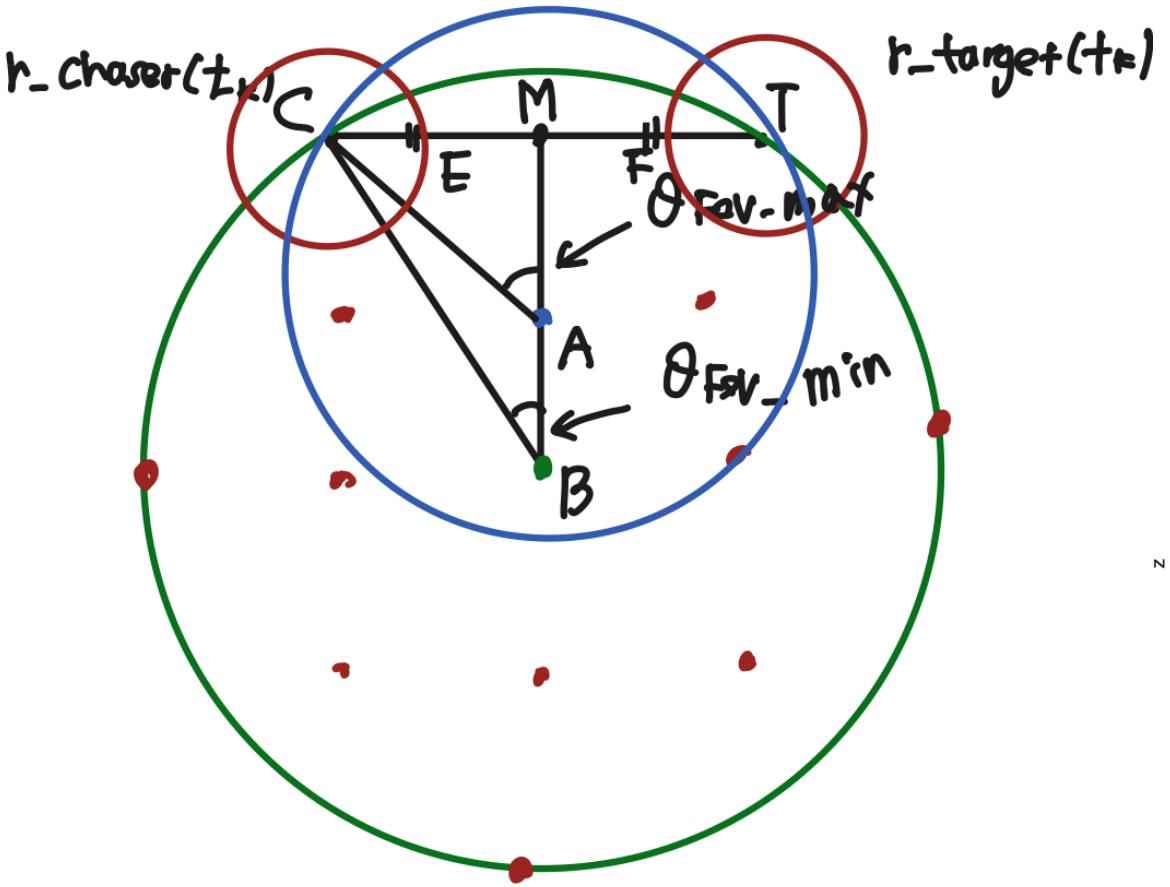
Problem Statement

- **Design Observer Satellite's Trajectory to capture both chaser and target in single FOV**
- **While minimizing delta-v for Observer Satellite**

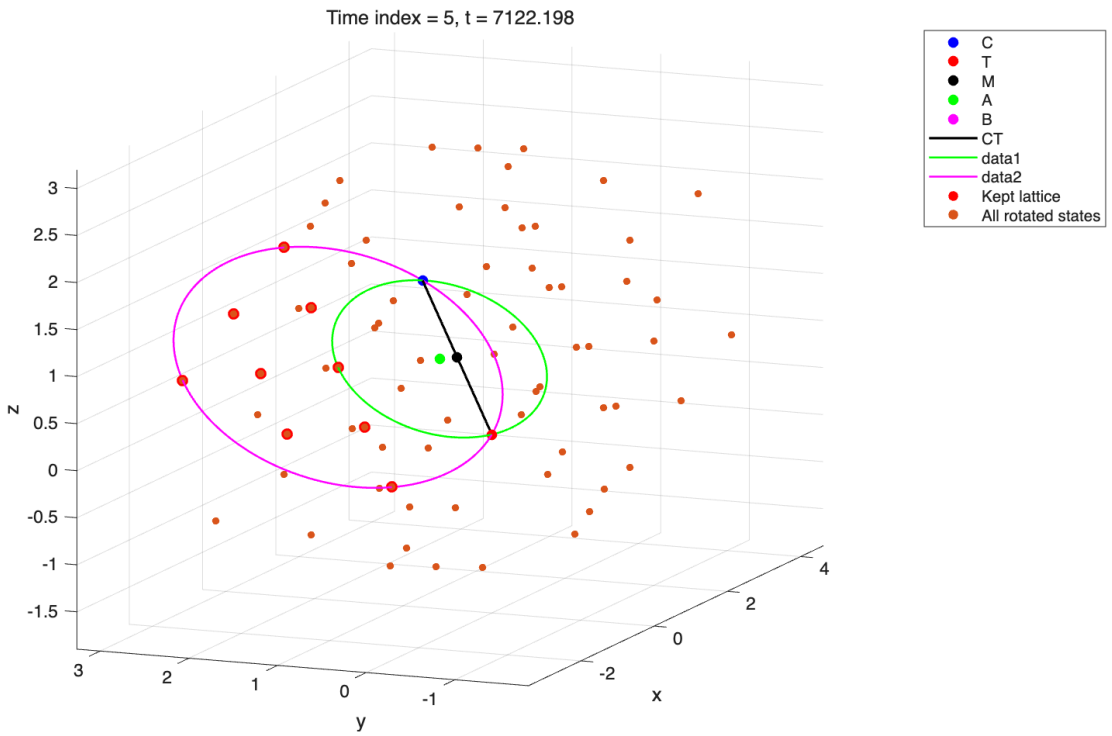


How to place observer satellite in given FOV condition?

- **We have to do Geometry**



- Red dot: available Points
- When placing this in this lattice points, the Observer will capture caser and target in given FOV condition
- Rotate lattice with respect to C,T will make donut space



MDP Formulation

We define a time-indexed MDP over a finite horizon $k = 0, 1, \ldots, T$.

State Definition

The state at time k is defined as:

$$s_k = (x_k, y_k, z_k)$$

To incorporate trajectory continuity, the system is modeled with augmented state:

$$\tilde{s}_k = (s_{k-1}, s_k)$$

This results in a second-order Markov structure.

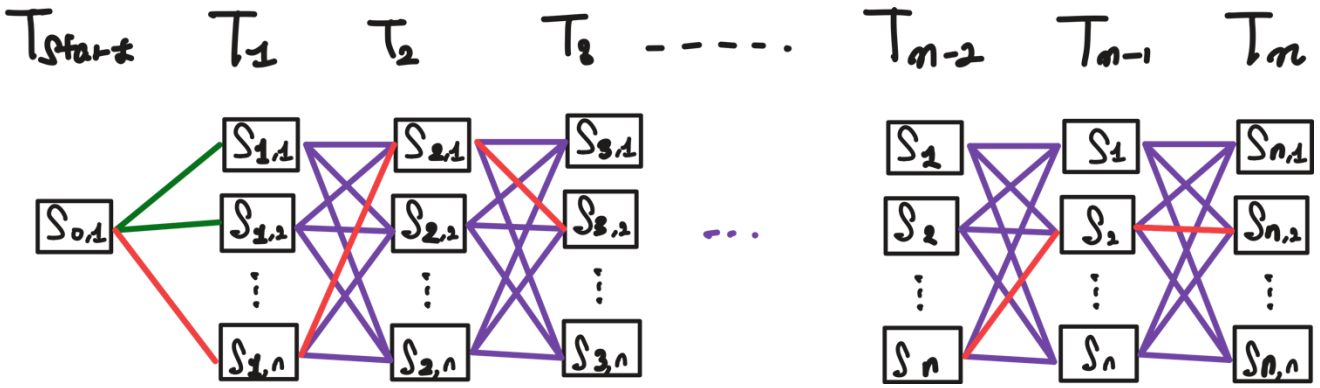
Action Definition

The action corresponds to selecting the next state:

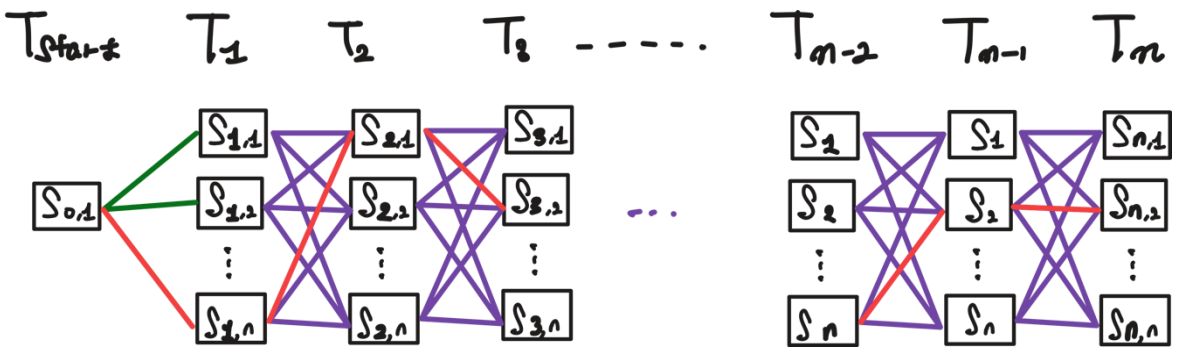
$$a_k = s_{k+1}$$

Thus, transitions are defined as:

$$(s_{k-1}, s_k) \to (s_k, s_{k+1})$$



MDP Formulation



Deterministic State Transition vis C-W dynamics

The state transition is governed by orbital mechanics through a C-W dynamics:

$$\ddot{x} - 2n\dot{y} - 3n^2 x = u_x,$$
$$\ddot{y} + 2n\dot{x} = u_y,$$
$$\ddot{z} + n^2 z = u_z,$$

$$(v_1, v_2) = f(r_1, r_2, \Delta t)$$
$$r_2 = \Phi_{rr} r_1 + \Phi_{rv} v_1$$
$$v_1 = \Phi_{rv}^{-1}(r_2 - \Phi_{rr} r_1)$$
$$v_2 = \Phi_{vr} r_1 + \Phi_{vv} v_1$$

Reward Function

The reward function enforces dynamical consistency between consecutive trajectory segments.

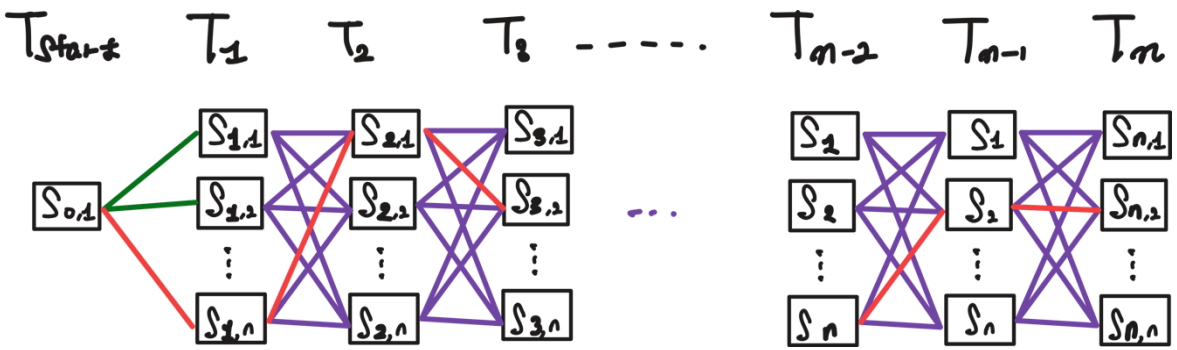
$$R(s_{k-1}, s_k, s_{k+1}) = -\|v_{\text{next}} - \hat{v}_{\text{prop}}\|$$

where:

- v_{next} is the velocity from the C-W dynamics solution between (s_k, s_{k+1})
- $\hat{v}_{\text{prop}}$ is the propagated velocity from previous segment (s_{k-1}, s_k)

This penalizes discontinuities in velocity and promotes physically feasible trajectories.

MDP Formulation



State Value function and Action Value Function

$$V(s_k, s_{k-1}) = \max_{s_{k+1}} q(s_{k-1}, s_k, s_{k+1})$$

$$q(s_{k-1}, s_k, s_{k+1}) = R(s_{k-1}, s_k, s_{k+1}) + V(s_k, s_{k+1})$$

Backward Dynamic Programming for getting optimal policy

The optimal value function is computed via backward recursion:

$$V(s_k, s_{k-1}) = \max_{s_{k+1}} [R(s_{k-1}, s_k, s_{k+1}) + V(s_k, s_{k+1})]$$

for $k = T-1, \ldots, 1$.

The optimal policy is defined as:

$$\pi^*(s_{k-1}, s_k) = \arg\max_{s_{k+1}} q(s_{k-1}, s_k, s_{k+1})$$

	time	previous_state	current_state	optimal_next_state	delta_t	state_value
1	0	1	1	34	593.52	NaN
2	593.52	1	34	34	593.52	-0.092355
3	1187	34	34	1	593.52	-0.070925
4	1780.5	34	1	38	593.52	-0.052136
5	2374.1	1	38	39	593.52	-0.039773
6	2967.6	38	39	14	593.52	-0.038613
7	3561.1	39	14	44	593.52	-0.031591
8	4154.6	14	44	38	593.52	-0.019682
9	4748.1	44	38	10	593.52	-0.01564
10	5341.6	38	10	2	593.52	-0.01085
11	5935.2	10	2	66	593.52	-0.0099955
12	6528.7	2	66	53	593.52	-0.0047658
13	7122.2	66	53	19	593.52	-0.0021582
14	7715.7	53	19	12	593.52	-0.0014469

MDP Structure

```
>> MDP

MDP =

  다음 필드를 포함한 struct:

    time_index_1: [1×1 struct]
    time_index_2: [1×1 struct]
    time_index_3: [1×1 struct]
    time_index_4: [1×1 struct]
    time_index_5: [1×1 struct]
    time_index_6: [1×1 struct]
    time_index_7: [1×1 struct]
    time_index_8: [1×1 struct]
    time_index_9: [1×1 struct]
   time_index_10: [1×1 struct]
   time_index_11: [1×1 struct]
   time_index_12: [1×1 struct]
   time_index_13: [1×1 struct]
   time_index_14: [1×1 struct]
   time_index_15: [1×1 struct]
```

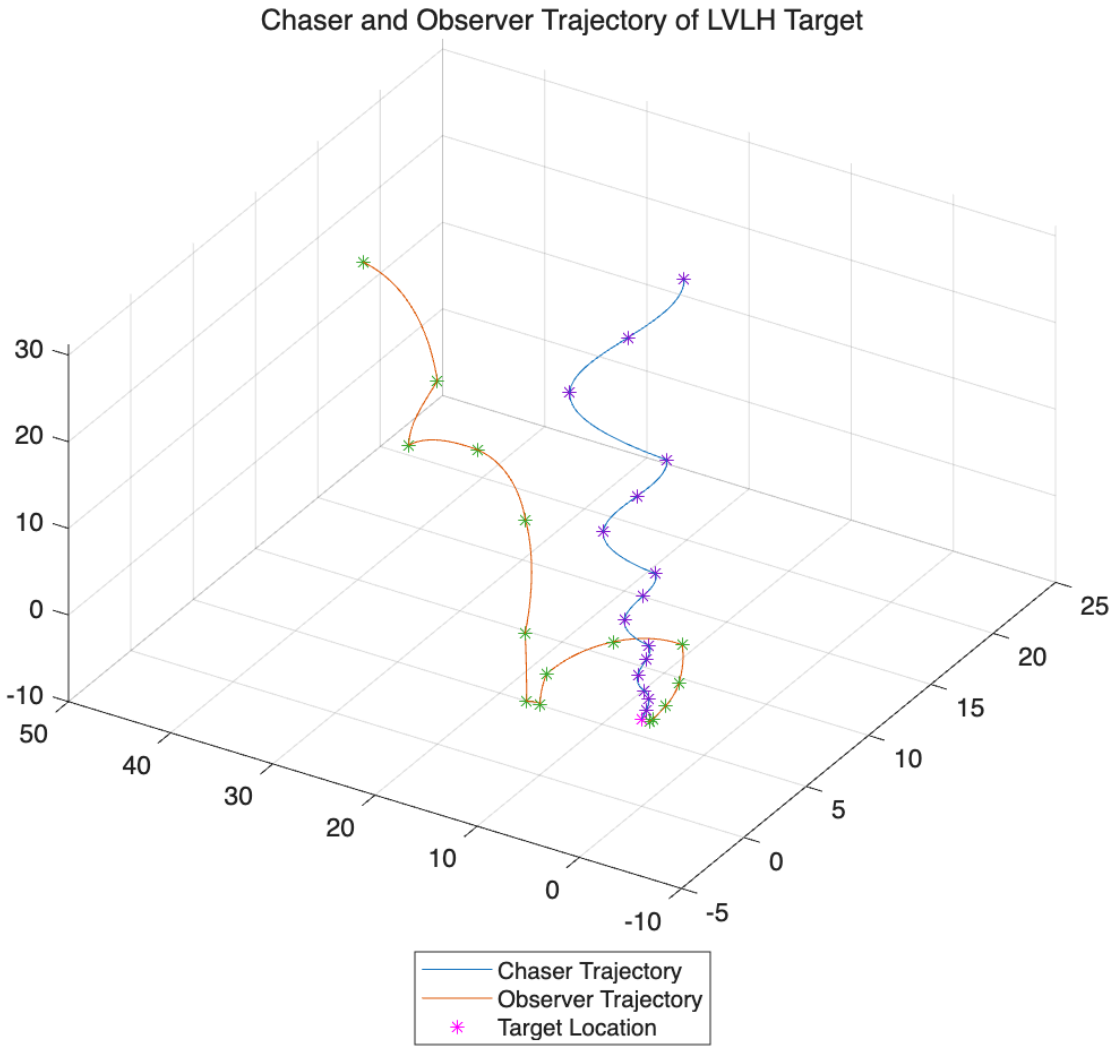
```
>> MDP.time_index_1

ans =

  다음 필드를 포함한 struct:

                 time_info: 0
    current_state_index_1: [1×1 struct]
    current_state_index_2: [1×1 struct]
    current_state_index_3: [1×1 struct]
    current_state_index_4: [1×1 struct]
    current_state_index_5: [1×1 struct]
    current_state_index_6: [1×1 struct]
    current_state_index_7: [1×1 struct]
    current_state_index_8: [1×1 struct]
   current_state_index_67: [1×1 struct]
   current_state_index_68: [1×1 struct]
   current_state_index_69: [1×1 struct]
   current_state_index_70: [1×1 struct]
   current_state_index_71: [1×1 struct]
   current_state_index_72: [1×1 struct]
      optimal_state_value: -0.060873
   optimal_starting_state: 31
```

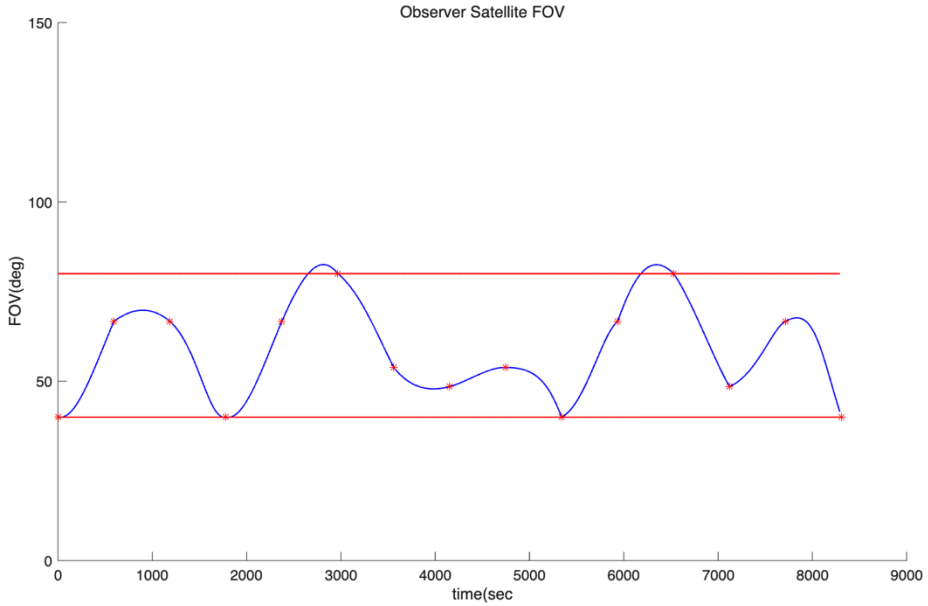
```
>> MDP.time_index_2.current_state_index_1

ans =

  다음 필드를 포함한 struct:

                 state_info: [3×1 double]
      next_state_index_1: [1×1 struct]
      next_state_index_2: [1×1 struct]
      next_state_index_3: [1×1 struct]
      next_state_index_4: [1×1 struct]
      next_state_index_5: [1×1 struct]
      next_state_index_6: [1×1 struct]
      next_state_index_7: [1×1 struct]
   previous_state_index_61: [1×1 struct]
   previous_state_index_62: [1×1 struct]
   previous_state_index_63: [1×1 struct]
   previous_state_index_64: [1×1 struct]
   previous_state_index_65: [1×1 struct]
   previous_state_index_66: [1×1 struct]
   previous_state_index_67: [1×1 struct]
   previous_state_index_68: [1×1 struct]
   previous_state_index_69: [1×1 struct]
   previous_state_index_70: [1×1 struct]
   previous_state_index_71: [1×1 struct]
   previous_state_index_72: [1×1 struct]
```

```
> MDP.time_index_2.current_state_index_1.previous_state_index

ns =

  다음 필드를 포함한 struct:

            arrival_velocity: [3×1 double]
          next_state_index_1: [1×1 struct]
          next_state_index_2: [1×1 struct]
          next_state_index_3: [1×1 struct]
          next_state_index_4: [1×1 struct]
          next_state_index_5: [1×1 struct]
          next_state_index_6: [1×1 struct]
          next_state_index_7: [1×1 struct]
         next_state_index_57: [1×1 struct]
         next_state_index_58: [1×1 struct]
         next_state_index_59: [1×1 struct]
         next_state_index_60: [1×1 struct]
         next_state_index_61: [1×1 struct]
         next_state_index_62: [1×1 struct]
         next_state_index_63: [1×1 struct]
         next_state_index_64: [1×1 struct]
                 state_value: -0.15199
          optimal_next_state: 5
   optimal_departure_velocity: [3×1 double]
                     delta_t: 593.52
```

```
>> MDP.time_index_2.current_state_index_1.previous_state_index_1.next_state_index_1

ans =

  다음 필드를 포함한 struct:

          reward: -0.068886
    action_value: -0.18411
```

RPOD Case - GlideSlope

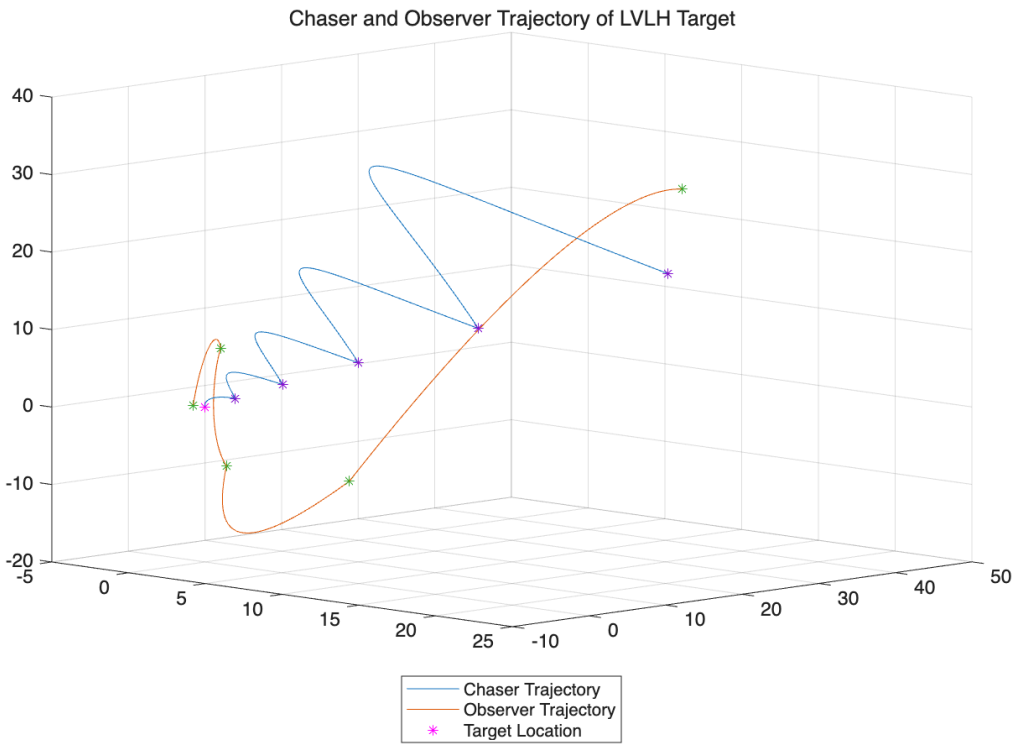


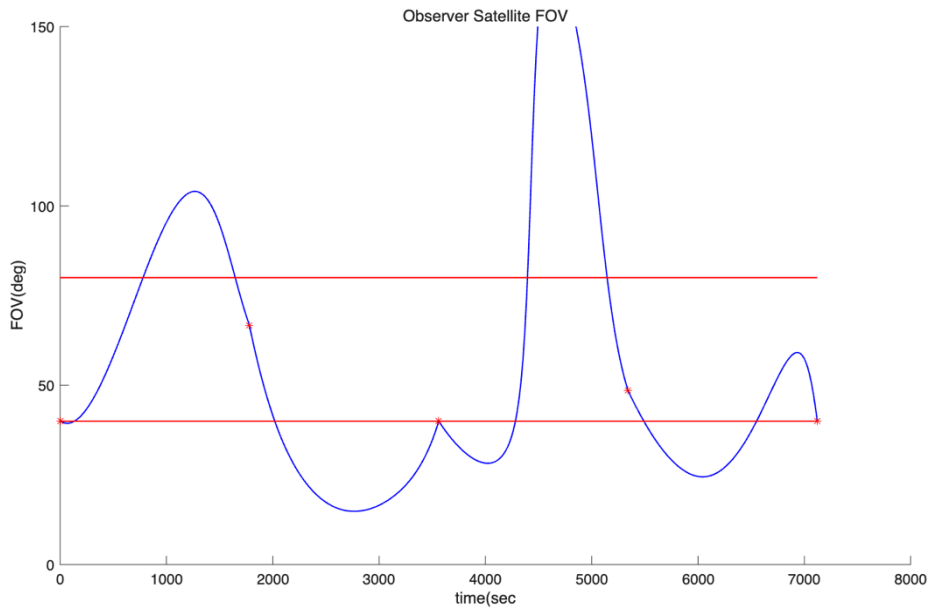


	time	state_value
1	0	NaN
2	593.52	-0.092355
3	1187	-0.070925
4	1780.5	-0.052136
5	2374.1	-0.039773
6	2967.6	-0.038613
7	3561.1	-0.031591
8	4154.6	-0.019682
9	4748.1	-0.01564
10	5341.6	-0.01085
11	5935.2	-0.0099955
12	6528.7	-0.0047658
13	7122.2	-0.0021582
14	7715.7	-0.0014469

RPOD Case – Glideslope

5 Impulse

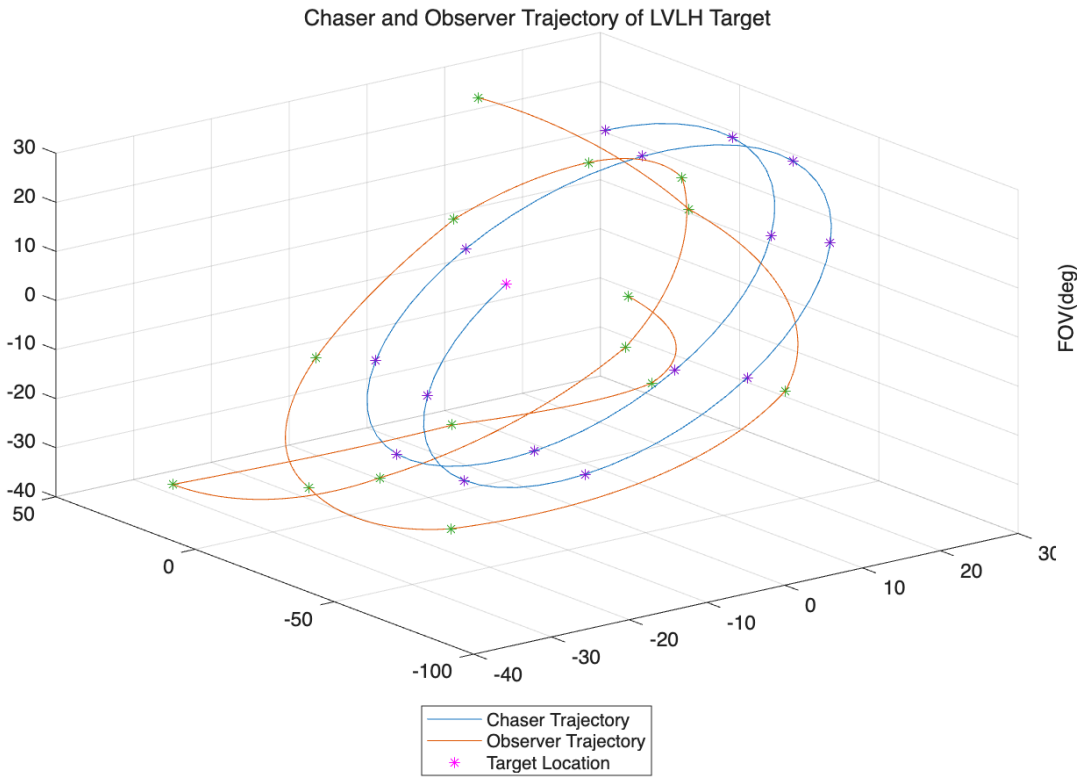


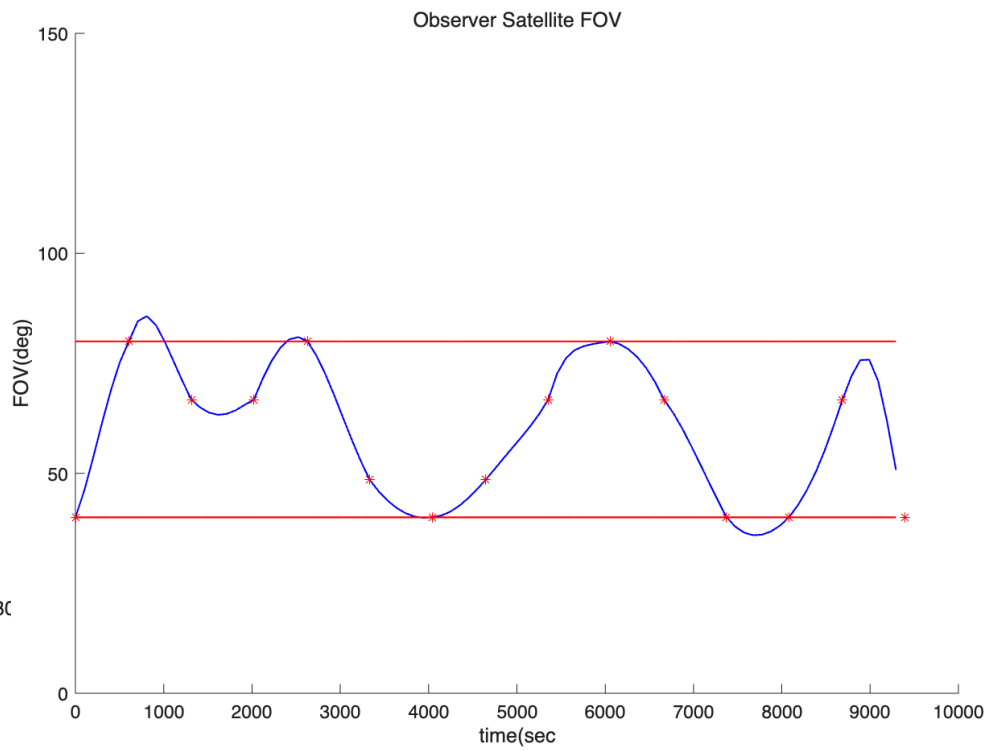


	state_value
1	NaN
2	-0.025122
3	-0.012172
4	-0.001136

RPOD Case: Two Impulse

14 Impulse

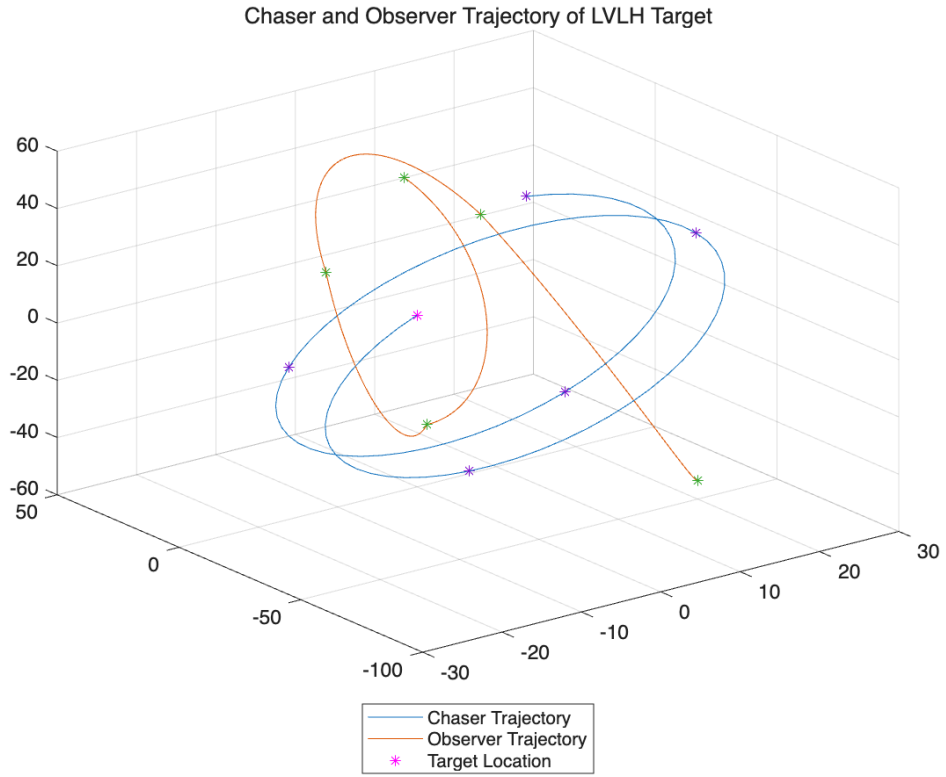


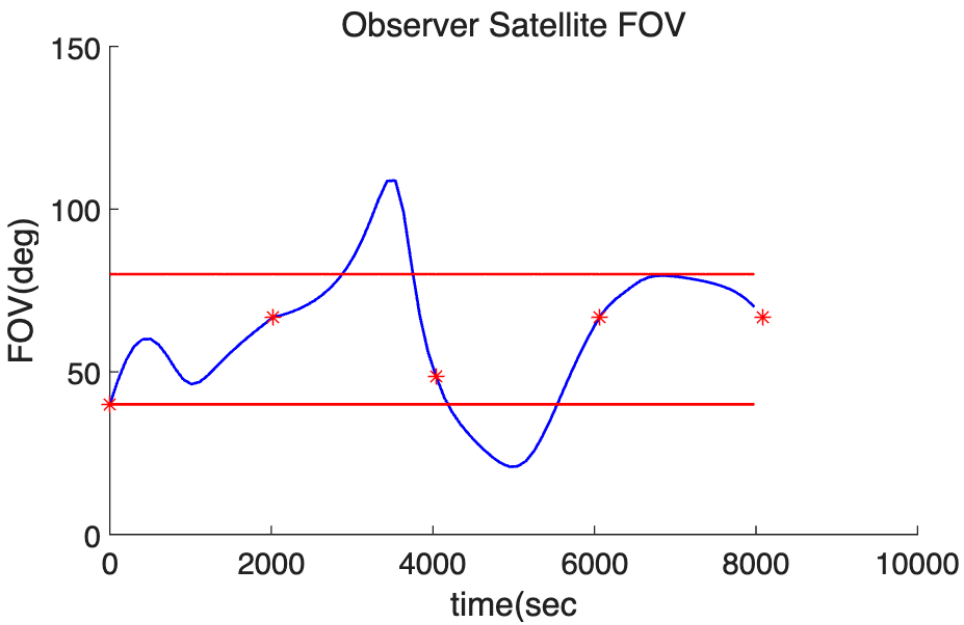


	state_value
1	NaN
2	-0.26887
3	-0.24973
4	-0.23417
5	-0.18481
6	-0.17643
7	-0.1657
8	-0.15519
9	-0.15032
10	-0.12488
11	-0.10994
12	-0.093588
13	-0.018272
14	-0.0078142

RPOD Case: Two Impulse

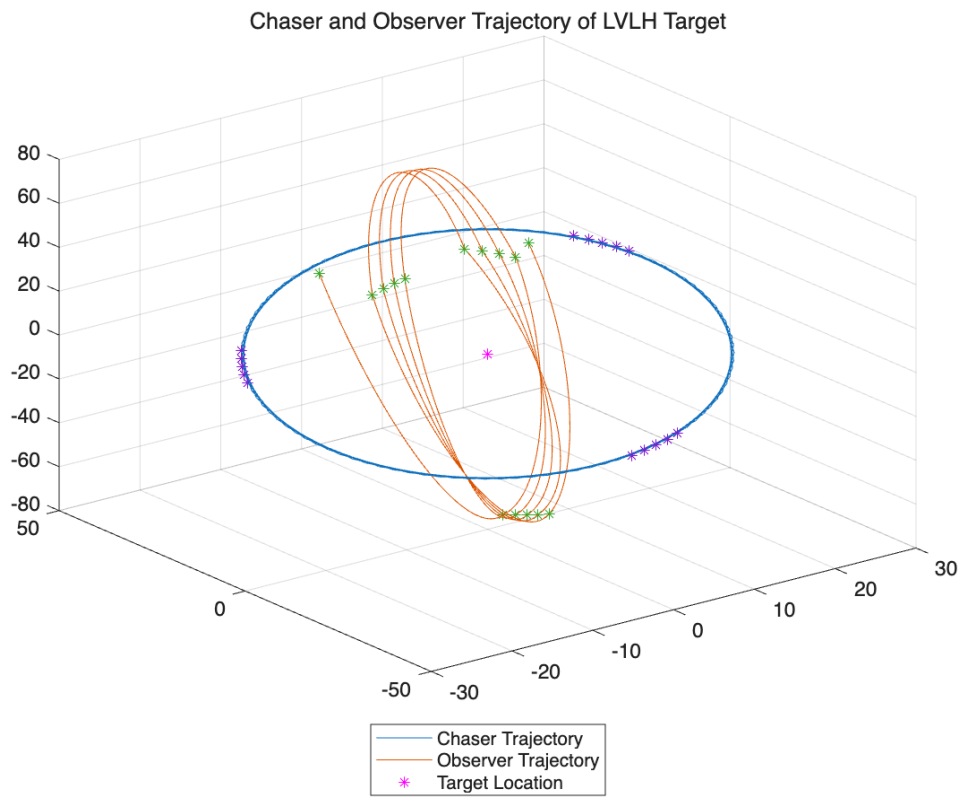
5 Impulse

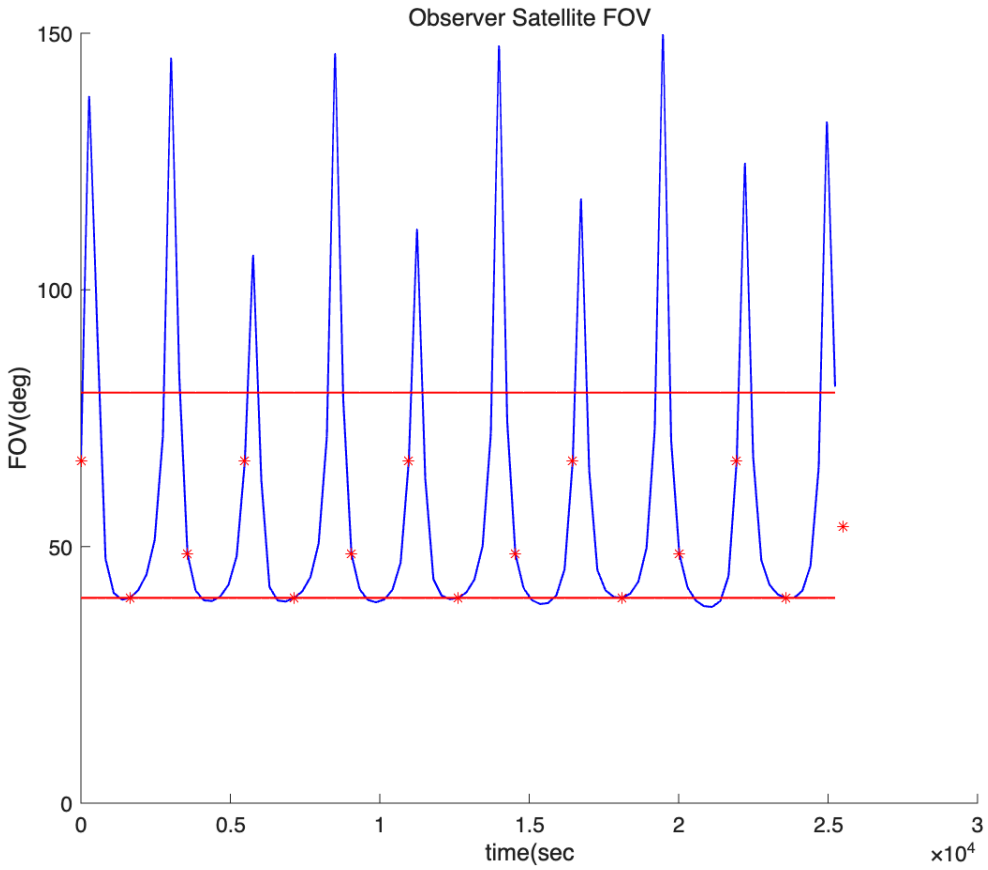




	state_value
1	NaN
2	-0.037278
3	-0.017254
4	-0.009449

RPOD Case: CircumNav





	state_value
1	NaN
2	-0.061683
3	-0.05887
4	-0.054917
5	-0.046754
6	-0.040485
7	-0.037006
8	-0.032447
9	-0.027418
10	-0.023309
11	-0.019716
12	-0.015506
13	-0.010085
14	-0.0038141

The University of Texas at Austin
Aerospace Engineering
and Engineering Mechanics
Cockrell School of Engineering